

Multi-Agentive AI-Native Autonomous Intelligent RAN

Complete Implementation Guide

Author: Swetha Kerahalli Organization: Nokia MI RRD AS Algo Innov

Dashboard: <http://localhost:8080/dashboard>

This project implements an end-to-end Autonomous Intelligent RAN platform with 17 AI-driven agents, a Digital Twin, REST API, live dashboard, Nokia MCP knowledge integration, and closed-loop optimization aligned with 3GPP Rel-18/19, O-RAN Alliance, a

Metric Before After Change

Throughput (API demo) 10.0 Mbps 19.07 Mbps +90.7%

Total Power 2800 W 1680 W -40.0%

Security Detection (multi-agent) 8.7% 100% +91.3 pp

scheduler, resource, mobility, security, energy

air_interface, digital_twin, spectrum, healing

Validate/Control 7 cells, 50 UEs

3. End-to-End Workflow (8 Phases)

Phase Script Input Output

1. Dataset Generation ``scripts/generate_datasets.py`` ``config/system_config.json`` 6 CSV files in ``data/datasets/``

3. Agent Training ``scripts/train_validate_test_agents.py`` CSV datasets ``outputs/models/*.joblib``

5. API Invocation ``scripts/invoke_api_demo.py`` ``POST /api/agents/run`` Before/after KPI JSON

7. Visualizations ``scripts/generate_visualizations.py`` CSV + models ``outputs/plots/`` (54+ plots)

Full pipeline: ``python scripts/run_end_to_end.py``

4. Generated Datasets (CSV)

All datasets are synthetic, 3GPP-aligned, seed=42.

`mobility\_traces.csv` 5,000 velocity_mps, rsrp_dbm, handover_pending mobility, channel_estimation

`energy\_metrics.csv` 3,500 power_consumption_w, cell_utilization, sleep_state energy, resource, digital_twin

`handover\_events.csv` 2,000 source_cell, target_cell, ho_success mobility

Export inventory: `python scripts/export\_e2e\_csv.py` `outputs/reports/dataset\_inventory.csv`

All 17 agents are AI-driven (sklearn GradientBoosting/RandomForest/MLP or LLM).

Agent Model Training Data Features API Action Type

`scheduler GradientBoosterRegressor ran_kpi cqi, sinr_db, buffer, latency, mcs, prb `schedule` prb_assignment, mcs`

mobility GradientBoostingClassifier mobility_traces velocity, rsrp, handover_pending `mobility` handover recommendation

qos GradientBoostingClassifier ran_kpi latency, throughput, packet_loss, prb `qos` priority_boost per slice

qoe GradientBoostingRegressor ran_kpi throughput, latency, packet_loss `qoe` MOS estimate, optimization

Agent Model Training Data API Action Type

channel_estimation MLRegressor mobility + ran_kpi `channel\_estimation` csi_accuracy

csi GradientBoosting + PCA ran_kpi `csi` compression_ratio, predicted throughput

5.3 Platform Agents

spectrum GradientBoostingClassifier security_events `spectrum` reallocate, bandwidth_mhz

knowledge LLM + Knowledge Graph KB JSON RCA, policy references

5.4 Super Agent

- Weights: 17 agents with weighted utility function from `config/kpis.json`


```
python scripts/train_validate_test_agents.py
```

1. Load CSV dataset (see Section 4)

3. ``agent.save(outputs/models/{name}_agent.joblib)``

5. Super Agent integration test all 17 agents approved

- scheduler R=0.13 resource R=0.999 mobility acc=1.0 security acc=1.0

- beamforming R=0.999 air_interface R=0.13 digital_twin R=0.999

- Super Agent utility: 5.40

Base URL: `http://localhost:8080`

Method	Endpoint	Description
--------	----------	-------------

GET `/api/health` Service health + agent list

GET `/api/kpi/current` Live KPI snapshot

GET `/api/kpi/targets` 9 target KPIs vs goals

GET `/api/e2e/summary` Full E2E implementation results

POST `/api/agents/run` Run all agents + apply to twin

PUT `/api/ran/ues/{id}` Update UE CQI/throughput/latency

POST `/api/closed-loop/run` ObserveActLearn loop

7.2 Agent Run Flow (POST /api/agents/run)

"intent": "Optimize throughput, reduce latency, improve energy",

1. `RANStateStore.snapshot_before()` capture KPI baseline

3. `IntentAgent` parses intent via LLM

5. ``SuperAgent.validate_and_control()`` approve/reject/modify

7. `RANDigitalTwin.step()` simulate next state

9. Return `kpi\_before`, `kpi\_after`, `parameter\_updates`, `super\_agent\_decision`

8.1 Knowledge Base Files

`oran\_references.json` O-RAN Alliance Architecture, E2SM-KPM, closed-loop

`telecom\_ontology.json` Project ontology Entities, threats, relationships

`feature\_store\_manifest.json` Builder Feature groups per agent

MCP Server Tools Purpose

user-system-insights `ask`,`search\_specs`,`read\_spec\_requirement` 3GPP, CFAM, NIDD, O-RAN specs

Refresh index: `python scripts/fetch\_external\_knowledge.py`

9. Dashboard

URL: <http://localhost:8080/dashboard>

1. Target KPIs 9 goals from `config/kpis.json` with progress bars

3. Slice KPIs eMBB, URLLC, mMTC breakdown

5. KPI Trend throughput/latency time series

7. Nokia Knowledge Base MCP source status

Start server: ``python scripts/run_api_server.py``

10. O-RAN Deployment

14 xApps configured in `config/oran\_config.json`:

4 rApps: analytics, policy_management, federated_training, ai_model_management

File Purpose

`config/system\_config.json` 7 cells, 50 UEs, 3 slices (eMBB/URLLC/mMTC)

`config/kpis.json` 9 target KPIs + utility function weights

`config/llm\_config.json` Ollama/Nokia LLM settings

12. Output Artifacts

Path Description

`outputs/models/\*.joblib` 15 trained sklearn models

`outputs/reports/agent\_train\_validate\_test.json` Full train/validate/test report

`outputs/reports/api\_kpi\_before\_after.csv` KPI delta CSV

`outputs/reports/e2e\_pipeline\_phases.csv` Pipeline phase status

cd code_auto_RAN

```
pip install python-docx fpdf2
```

```
python scripts/generate_datasets.py
```

```
python scripts/train_validate_test_agents.py
```

```
python scripts/export_e2e_csv.py
```

python scripts/generate_documentation.py

```
python scripts/run_api_server.py
```

API demo (before/after KPI)

python scripts/invoke_api_demo.py

```
python scripts/run_end_to_end.py
```

14. Project Structure

code_auto_RAN/

data/

knowledge_base/ # 3GPP, O-RAN, Nokia MCP JSON

outputs/

plots/ # Visualizations

scripts/ # Runnable pipelines

agents/ # 17 AI agents + Super Agent

chatbot/ # LLM chatbot

orchestration/# Super Agent controller, closed loop

- 3GPP: TS 38.214 (scheduling), TS 38.331 (mobility), TS 28.104 (NDT), TS 28.554 (KPIs), TS 28.100 (autonomy levels)

- Nokia CFAM: OSS_FC_017307 (MRO), SR003080 (energy), SR001534 (self-healing)

